

Nexus Bank Virtual Assistant

A RAG-Based Conversational System for Retail Banking Q&A

Project Report

[Course / Codecademy Assessment]

March 7, 2026

Target Length: 8–15 Pages

Contents

1	Introduction	2
1.1	Problem Statement	2
1.2	Motivation	2
1.3	Why RAG instead of Fine-Tuning or Pure Generation?	2
2	Related Work	3
2.1	The Origins of RAG	3
2.2	Recent Advances in Retrieval and Agents	3
3	Chosen Use Case & Dataset Description	4
3.1	The Use Case	4
3.2	Dataset Source and Structure	4
3.3	Data Cleaning and Processing Steps	4
3.4	External Live Data (NBP API)	4
4	System Architecture	6
4.1	High-Level Flow	6
4.2	Architecture Diagram	6
5	Implementation Details	8
5.1	Data Ingestion & Preprocessing	8
5.2	Chunking Strategy	8
5.3	Embedding Model & Rationale	8
5.4	Vector Database & Indexing	8
5.5	Retrieval Method and Tools	10
5.6	LLM Choice & Generation Parameters	10
5.7	Prompt Engineering / Augmentation Strategy	10
6	Experiments & Evaluation	11
6.1	Qualitative Evaluation: Hallucination Reduction	11
6.2	Comparison: Pure LLM vs. RAG System	11
6.3	Quantitative Evaluation (RAGAS)	13
7	Challenges Faced & How Solved	13
8	Conclusion & Possible Extensions	14
8.1	Conclusion	14
8.2	Possible Extensions	14
9	References	14

1 Introduction

1.1 Problem Statement

The retail banking sector handles millions of customer inquiries daily. A significant portion of these queries revolves around standard informational topics: account types, maintenance fees, interest rates on term deposits, and foreign exchange (FX) rates. Traditionally, these inquiries are handled by human customer service representatives in call centres. However, maintaining large call centres is highly resource-intensive and often leads to long wait times for clients, especially outside of standard business hours.

While Large Language Models (LLMs) offer a promising avenue for conversational automation, deploying pure, unaugmented LLMs in the highly regulated financial sector poses severe risks. Pure generative language models are prone to “hallucinations”—generating fluent but factually incorrect information. In a banking context, quoting a non-existent account fee, inventing a promotional interest rate, or providing a wildly inaccurate currency exchange rate can lead to significant financial liability, regulatory fines, and a catastrophic loss of customer trust.

1.2 Motivation

The motivation behind this project is to build a robust, highly reliable virtual assistant for a fictional entity, Nexus Bank S.A. The system must maintain the conversational fluidity of modern LLMs while strictly adhering to a defined, approved knowledge base. By grounding the model’s responses in the official “General Terms and Conditions” document and fetching real-time data from authorized APIs, we can create an assistant that reduces operational load, improves 24/7 availability, and guarantees compliance with official banking policies. Furthermore, it ensures there is no financial advice given, no collection of passwords/PINs, and no access to private account balances.

1.3 Why RAG instead of Fine-Tuning or Pure Generation?

When designing a domain-specific LLM system, engineers typically choose between Pure Generation, Fine-Tuning, and Retrieval-Augmented Generation (RAG).

- **Pure Generation:** Relying solely on the parametric memory of a model is unviable. The model does not know the specific, proprietary rules of Nexus Bank, nor can it know today’s live exchange rates. It will inevitably hallucinate figures.
- **Fine-Tuning:** While fine-tuning injects domain knowledge into the model’s weights, it is an expensive and slow process. Banking Terms and Conditions can change frequently (e.g., updating an interest rate). Retraining a model every time a fee changes is impractical. Furthermore, fine-tuning inherently cannot solve the problem of real-time data retrieval, such as fetching the daily EUR to PLN exchange rate.
- **Retrieval-Augmented Generation (RAG):** RAG dynamically provides the LLM with the exact, up-to-date context it needs to answer a specific query. By splitting the official Terms and Conditions into a searchable vector database, and by giving the agent access to external API tools, we keep the LLM as a general reasoning engine. The knowledge is decoupled from the model.

Therefore, an Agentic RAG architecture was chosen as the most secure, scalable, and compliant solution for this use case.

2 Related Work

2.1 The Origins of RAG

The foundational architecture for Retrieval-Augmented Generation was formalized by Lewis et al. (2020). In their seminal paper, they proposed conditioning pre-trained sequence-to-sequence models on top- k documents retrieved from a massive external corpus using a dense retriever. This approach demonstrated that providing explicit, retrieved context drastically reduced hallucinations in knowledge-intensive NLP tasks and allowed the system’s memory to be updated without retraining the underlying neural network.

2.2 Recent Advances in Retrieval and Agents

Since the original RAG paper, the ecosystem has evolved rapidly, incorporating advanced retrieval techniques and autonomous agent behaviours:

1. **Dense Passage Retrieval (DPR):** Karpukhin et al. (2020) demonstrated that dense embeddings (mapping text to high-dimensional continuous vectors) significantly outperform traditional sparse retrieval methods like BM25 for semantic search. Modern RAG systems, including ours, rely on state-of-the-art embedding models (such as OpenAI’s `text-embedding-3-small`) to capture the deep semantic meaning of user queries and document chunks.
2. **Tool-Augmented LLMs (Agentic RAG):** Standard RAG is limited to static document retrieval. Schick et al. (2023) introduced the concept of teaching language models to use external tools via API calls. This has evolved into frameworks like LangChain and LangGraph, which allow LLMs to act as autonomous agents. An agent evaluates the user’s query and decides whether it needs to query a vector database or hit a live API (Gao et al., 2023). Our system heavily leverages this Agentic RAG paradigm to handle live NBP currency queries alongside document retrieval.
3. **Automated RAG Evaluation (RAGAS):** Evaluating generative systems has historically been subjective and difficult. Es et al. (2023) introduced RAGAS (Retrieval Augmented Generation Assessment), a framework that utilizes LLMs as evaluators to compute objective metrics such as Faithfulness, Answer Relevance, and Context Precision without requiring human-annotated ground truth for every query. This advancement allows for continuous, automated quality control of the RAG pipeline.

3 Chosen Use Case & Dataset Description

3.1 The Use Case

The project simulates a Virtual Assistant ("Nexus Assistant") deployed by a fictional retail bank operating in Poland: **Nexus Bank S.A.** The assistant is designed to interact with retail clients via a dual interface (Terminal and Streamlit Web UI), answering questions regarding account maintenance fees, savings interest rates, penalty rules, and foreign exchange procedures.

3.2 Dataset Source and Structure

Unlike general-domain RAG systems built on Wikipedia, this system is grounded in a specific, proprietary document.

- **Source:** A highly detailed PDF document titled *General Terms and Conditions of Retail Banking Services of Nexus Bank S.A.* located at `data/Nexus Bank Terms and Conditions.pdf`.
- **Size:** The document comprises 7 pages of dense legal and financial text, structured into 8 formal chapters (General Provisions, Account Types, Interest Rates, Foreign Exchange, Fees, Security, Complaints, and Final Provisions).
- **Format:** Standard PDF format.

3.3 Data Cleaning and Processing Steps

The ingestion process prioritizes maintaining the structural integrity of the text.

1. **Extraction:** The text is extracted using LangChain's PyPDFLoader. This loader reads the document page by page, preserving the sequence of the text.
2. **Cleaning:** No custom, aggressive cleaning is applied in the code. Financial documents rely heavily on exact phrasing, numbers, and punctuation. Stripping characters aggressively risks destroying crucial financial data (e.g., altering "5.50%" to "550").
3. **Chunking:** The raw extracted pages are immediately passed to the `RecursiveCharacterTextSplitter` to create mathematically manageable vectors.

3.4 External Live Data (NBP API)

To demonstrate handling dynamic data that cannot be stored in a static PDF, the system integrates with the National Bank of Poland (NBP). The Terms and Conditions explicitly state that the Bank uses NBP rates without a proprietary spread.

When triggered, the tool makes an HTTP GET request to the NBP Web API. Below is a representation of the JSON payload returned by the API for the Euro (EUR), which the agent parses in real-time:

```
{
  "table": "A",
  "currency": "euro",
  "code": "EUR",
  "rates": [
    {
      "no": "045/A/NBP/2026",
      "effectiveDate": "2026-03-05",
```

```
    "mid": 4.3150  
  }  
]  
}
```

Listing 1: Sample Response Structure from NBP API

4 System Architecture

4.1 High-Level Flow

The architecture is built around an agentic workflow managed by **LangGraph**. Instead of a linear sequential chain (User → Retrieve → Generate), the system utilizes a cyclic graph structure. The LLM acts as the central reasoning node, capable of executing multiple actions (tool calls) iteratively before returning a final answer to the user.

1. **Input:** The user submits a query via a terminal interface (`main.py`) or a Streamlit Web UI (`streamlit_app.py`).
2. **Agent Evaluation:** The query is wrapped in a message history and passed to the LangGraph Agent. The state is maintained using an `InMemorySaver` checkpoint.
3. **Tool Routing:** The LLM evaluates the query against its system prompt. If the query requires knowledge of banking rules, it calls the `retrive_from_vector_db` tool. If the query involves live currency rates, it binds the `currency_exchange` tool.
4. **Execution & Observation:** The tools execute the Python functions. The vector DB returns chunked text; the API returns JSON data. These observations are added to the graph's state.
5. **Generation:** The LLM reviews the observations. If it has sufficient information, it streams the final, compliant response back to the client via the `run_agent_turn` utility.

4.2 Architecture Diagram

Below is a structured representation of the system's components and data flow (as outlined in the project's README):

```
flowchart LR
    subgraph client [Client]
        CLI[Terminal main.py]
        Web[Streamlit UI]
    end
    subgraph app [Application]
        Agent[LangGraph Agent]
        Tools[Tools]
    end
    subgraph tools_detail [Tools]
        NBP[NBP API currency]
        RAG[Vector DB retrieval]
    end
    subgraph data [Data]
        PDF[Terms PDF]
        Pinecone[Pinecone]
        NBPAPI[NBP API]
    end

    CLI --> Agent
    Web --> Agent
    Agent --> Tools
    Tools --> NBP
```

Tools --> RAG
PDF --> Pinecone
RAG --> Pinecone
NBP --> NBPAPI

5 Implementation Details

5.1 Data Ingestion & Preprocessing

The ingestion pipeline is designed to populate the vector database. It utilizes LangChain's PyPDFLoader to parse the binary PDF file.

```
def initialize_vector_db():
    loader = PyPDFLoader("data/Nexus Bank Terms and Conditions.pdf")
    docs = loader.load()
    vector_store = get_vector_store()
    text_split = RecursiveCharacterTextSplitter(
        chunk_size=800, chunk_overlap=200, add_start_index=True
    )
    all_splits = text_split.split_documents(docs)
    vector_store.add_documents(all_splits)
    return vector_store
```

Listing 2: Data Ingestion in vector_db.py

5.2 Chunking Strategy

Choice: RecursiveCharacterTextSplitter

Parameters: chunk_size=800, chunk_overlap=200, add_start_index=True.

Rationale: Financial documents are highly structured. The *Recursive* splitter attempts to split on natural boundaries first (paragraphs, then sentences, then spaces) before hard-cutting at 800 characters. A size of 800 characters provides enough surrounding context for the LLM to understand a rule (e.g., conditions for waiving a card fee). The 200-character overlap acts as a safety net, ensuring that concepts spanning across a split boundary are captured in both adjacent chunks.

5.3 Embedding Model & Rationale

Choice: OpenAI text-embedding-3-small (1536 dimensions).

Rationale: This model was chosen for its optimal balance of semantic retrieval performance and cost-efficiency. The 1536-dimensional vector space provides a highly granular representation of the text, allowing the cosine similarity metric to distinguish between closely related concepts.

5.4 Vector Database & Indexing

Choice: Pinecone (Serverless Architecture on AWS).

The implementation includes robust error handling. If the specified index (codecademy-assesment) does not exist, the code automatically creates it using the ServerlessSpec before inserting the embedded chunks.

```
def get_vector_store():
    try:
        index = pc.Index("codecademy-assesment")
        return PineconeVectorStore(index=index, embedding=embeddings)
    except Exception as e:
        index = pc.create_index(
            name="codecademy-assesment",
            dimension=1536,
            spec=ServerlessSpec(cloud="aws", region="us-east-1"),
            metric="cosine",
```

```
)  
return PineconeVectorStore(index=index, embedding=embeddings)
```

Listing 3: Pinecone Initialization Logic

5.5 Retrieval Method and Tools

The system employs **Cosine Similarity Search**. When the agent triggers the retrieval tool, it is configured to return the Top- K ($k = 3$) most similar chunks.

Additionally, the `currency_exchange` tool interacts with the public NBP API.

```
@tool(
    "currency_exchange",
    description=("Used to search for a currency exchange rate by its ISO 4217 code."),
    parse_docstring=True,
)
def check_currency_rate(currency_code: str) -> dict:
    r = requests.get(
        f"http://api.nbp.pl/api/exchangerates/rates/A/{currency_code}?format=json"
    )
    if r.status_code == 200:
        return r.json()
    else:
        raise Exception(f"Invalid service status: {r.status_code}")

@tool
def retrieve_from_vector_db(query: str):
    vector_store = get_vector_store()
    return vector_store.similarity_search(query=query, k=3)
```

Listing 4: Tools Implementation in tools.py

5.6 LLM Choice & Generation Parameters

Primary Model: gpt-5-mini (OpenAI) with temperature=0.5.

Secondary Model: llama3.2:3b via Ollama.

Rationale: We utilize a modern reasoning model configured with a moderate temperature. In a banking context, hallucination must be avoided, but the conversational flow must remain natural. The system supports switching to a local, open-weights model via the Streamlit UI (streamlit_app.py) using a dictionary mapping: `MODELS = {"openai": llm, "ollama": llm_2}`.

5.7 Prompt Engineering / Augmentation Strategy

The core intelligence of the agent is guided by an extensive System Prompt (system_msg) in `agent.py`. It explicitly sets constraints and mathematical procedures:

```
3. TOOL USAGE: CURRENCY EXCHANGE
Trigger Condition: Whenever a user asks about converting currencies, current
exchange rates, or the cost of a transaction in a foreign currency, you MUST use
this tool.
Input Parameters: The tool requires an ISO 4217 three-letter currency code (e.g., "
EUR", "USD").
Mandatory Commission Calculation: According to the Terms and Conditions, every
foreign exchange transaction incurs a flat operational commission of 2.50 PLN.

4. STRICT CONSTRAINTS AND COMPLIANCE
No Financial Advice: You are an informational assistant, not a financial advisor.
No Authentication Processing: Do not ask users for their passwords, PIN codes, full
debit card numbers, or SMS OTPs.
```

Listing 5: Key Directives from the System Prompt

6 Experiments & Evaluation

Evaluation is the most critical component of deploying an enterprise RAG system. We assessed the system using both qualitative query testing and automated quantitative metrics using the RAGAS framework.

6.1 Qualitative Evaluation: Hallucination Reduction

To demonstrate the effectiveness of RAG, we present a series of test queries. The table below illustrates how the agent correctly utilizes retrieved context and API tools to formulate accurate answers.

Table 1: Qualitative Query Examples and RAG Behaviour

Test Case 1	Simple Knowledge Retrieval
User Query	"What are the account types offered by Nexus Bank?"
Tool Action	<code>retrieve_from_vector_db("account types")</code>
Retrieved Context	<i>...Bank offers Standard Account, Premium Account, and Savings Plus Account...</i>
Final Answer	"Nexus Bank offers three main account types for retail clients: Standard Account, Premium Account, and Savings Plus. For full details on fees and conditions, please refer to the Terms and Conditions..."

Test Case 2	Composite Logic (Retrieval + API + Math)
User Query	"What is the current EUR exchange rate? How much would 100 EUR cost me in PLN?"
Tool Actions	1. <code>retrieve_from_vector_db("foreign exchange commission")</code> 2. <code>check_currency_rate("EUR")</code>
Final Answer	"The current NBP average rate for EUR is [X.XXXX] PLN. For 100 EUR: Conversion: $100 * [X.XXXX] = [XXX.XX]$ PLN. Bank commission (per Terms): 2.50 PLN. Total: $[XXX.XX] + 2.50 = [XXX.XX]$ PLN."

6.2 Comparison: Pure LLM vs. RAG System

Subjecting a standalone generative LLM to the exact same questions without the Pinecone knowledge base or NBP tools reveals severe limitations.

Table 2: Comparison of Pure Generative LLM vs. Agentic RAG

Question	Pure LLM (No RAG/Tools)	Nexus Bank RAG System
<i>"How much does a 6-month term deposit pay?"</i>	"Nexus Bank currently offers around 3.5% to 5% on a 6-month deposit depending on promotions." (Guessing / Hallucination)	"The Bank offers a fixed rate of 4.50% per annum for a 6-Month Term Deposit." (Grounded)
<i>"What is the exact NBP exchange rate for USD right now?"</i>	"I don't have access to real-time data, but the USD to PLN rate is generally around 4.00 PLN." (Cannot fetch data)	"The current exact NBP average exchange rate for USD is 3.98 PLN." (Live API call)

6.3 Quantitative Evaluation (RAGAS)

We implemented an automated evaluation pipeline (`ragas_eval.py`) utilizing the **RAGAS framework**. A test dataset consisting of user inputs, reference ground-truths, generated responses, and retrieved contexts was constructed.

The evaluation utilizes `gpt-5-mini` as the evaluator LLM and operates asynchronously to speed up the assessment of multiple metrics: Faithfulness, FactualCorrectness, ContextRecall, and ContextPrecision.

```
async def evaluation():
    final_score = []
    for i, result in enumerate(results):
        score = await faithfulness_score.ascore(
            user_input=result["user_input"],
            response=result["response"],
            retrieved_contexts=result["retrived_contexts"],
        )
        final_score.append({f"faithfulness_{i}": score.value})
    # ... loop repeats for Context Recall, Precision, and Factual Correctness
    return final_score

result = asyncio.run(evaluation())
```

Listing 6: RAGAS Asynchronous Evaluation Loop Excerpt

By tracking these objective metrics, we can quantify hallucination reduction. High Faithfulness scores indicate that the model's responses are tightly bound to the retrieved Pinecone chunks, while high Context Recall validates the 800/200 chunking strategy's effectiveness at finding the right clauses.

7 Challenges Faced & How Solved

Building an agentic RAG system presents unique technical challenges:

- 1. Index Initialization Failures:** Initially, running the app without an existing Pinecone index caused exceptions. *Solution:* We implemented a robust try-except block in `vector_db.py`. If `pc.Index` fails to find `codecademy-assesment`, it automatically calls `pc.create_index` with the required `ServerlessSpec` before inserting documents.
- 2. Tool Argument Handling:** Users might type "Euros" or "Dollars". Passing these strings to the NBP API results in HTTP errors, as it strictly requires ISO codes. *Solution:* Resolved via Prompt Engineering. The System Prompt explicitly instructs the LLM: *"If the user asks about 'Euros', translate this to 'EUR' before calling the tool."*
- 3. Asynchronous Event Loop Conflicts:** Running the RAGAS evaluation framework caused event loop issues because LangGraph, Pinecone, and RAGAS async functions interact heavily. *Solution:* We isolated the RAGAS metrics computation within an `async def evaluation()` block and executed it cleanly using `asyncio.run(evaluation())` after data processing.
- 4. Streaming Output in Web UI:** Displaying agent responses word-by-word in Streamlit is complex when using a graph structure. *Solution:* We created a dedicated `run_agent_turn` generator in `chat_utils.py` that uses the `stream_mode="messages"` flag, filtering for AI content to yield chunks to Streamlit's `st.write_stream`.

8 Conclusion & Possible Extensions

8.1 Conclusion

We successfully designed, implemented, and evaluated a highly robust RAG-based virtual assistant for Nexus Bank S.A. By strictly grounding the language model in a vector database containing the Bank's Terms and Conditions, and augmenting its capabilities with live API tools, we solved the critical issue of LLM hallucination in a financial context. The dual-interface application (Terminal and Streamlit) provides high accessibility, while the rigorous system prompt ensures compliance regarding financial advice and user data security.

8.2 Possible Extensions

As outlined in the project README, several enhancements could be implemented for future production releases:

- **Expanded Knowledge Base:** Integrating FAQ documents, internal procedural manuals, and promotional product sheets into the Pinecone index to broaden the assistant's capabilities.
- **Authentication and Read-Only Core Banking Access:** In a secure, logged-in state, providing the agent with tools to perform read-only queries on the user's actual database (e.g., mock "balance"), allowing for highly personalized assistance.
- **Continuous Integration for RAGAS:** Integrate the `ragas_eval.py` script into a CI pipeline or a scheduled job for periodic evaluation reports when documents are updated.
- **Local Vector Store Fallback:** Integrating a local store like Chroma as an alternative to mitigate the dependency on Pinecone's cloud infrastructure.

9 References

Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). *RAGAS: Automated Evaluation of Retrieval Augmented Generation*.

LangChain. *LangChain Documentation: Agents and Tools*. <https://python.langchain.com/docs/modules/agents/>

LangGraph. *LangGraph Official Documentation*. <https://langchain-ai.github.io/langgraph/>

Narodowy Bank Polski. *NBP Web API: Exchange Rates*. <http://api.nbp.pl/>

OpenAI. *API Reference: Embeddings and Models*. <https://platform.openai.com/docs/>

Pinecone Systems. *Pinecone Vector Database Documentation*. <https://docs.pinecone.io/>